

Vedlegg D. Sentral modellkode for den operasjonelle rute- og lagermodellen. Utdraget under viser kjernen i den lineære kostnadsminimeringsmodellen som er beskrevet i kapittel 6, slik den er implementert i `run_route_inventory_model.py`. Hjelpfunksjoner for innlesing av rådata, filskriving, figurgenerering og resultatformatering er utelatt for lesbarhetens skyld; den fullstendige kildekoden, sammen med skript for datavask, sensitivitetsanalyse og validering, ligger i prosjektmappen `006_analysis`. Koden er gjengitt med pseudonymiserte havnekoder (P001-P004) på samme måte som i resten av rapporten.

D.1 Parametere og datastrukturer. Faktoren for ekstern/ukjent proxypris og datastrukturene som holder en voyage-etappe, en kjøpsmulighet og prisgrunnlaget.

```
from scipy.optimize import linprog
```

```
DEFAULT_EXTERNAL_PRICE_MULTIPLIER = 1.25
```

```
SENSITIVITY_MULTIPLIERS = (1.10, 1.25, 1.50)
```

```
@dataclass(frozen=True)
```

```
class Leg:
```

```
    vessel_file_id: str
```

```
    vessel_class: str
```

```
    voyage_number: str
```

```
    leg_sequence: int
```

```
    period_month: str
```

```
    from_port: str
```

```
    to_port: str
```

```
    available_ports: tuple[str, ...]
```

```
    consumption: float
```

```
    rob_start: float
```

```
    rob_end: float | None
```

```
    data_quality_flag: str
```

```
@dataclass(frozen=True)
```

```
class PurchaseOpportunity:
```

```
    leg_index: int
```

```

port: str
price: float
price_source: str

```

```
@dataclass(frozen=True)
```

```
class PriceData:
```

```

    exact_prices: dict[tuple[str, str], float]
    port_average_prices: dict[str, float]
    external_price: float
    external_price_multiplier: float

```

D.2 Prisgrunnlag og proxypris. Vektet historisk havnesnitt beregnes per modellhavn, og proxykostnaden for eksternt/ukjent bunkring settes til faktoren ganger høyeste havnesnitt.

```
def load_price_data(external_price_multiplier: float = 1.25) -> PriceData:
```

```

    exact_prices: dict[tuple[str, str], float] = {}
    port_totals: dict[str, dict[str, float]] = {}
    for row in read_csv(PRICE_CSV):
        port = row["port"]
        month = row["delivery_month"]
        qty = float(row["total_qty"])
        price = float(row["weighted_avg_price"])
        exact_prices[(month, port)] = price
        if port not in port_totals:
            port_totals[port] = {"qty": 0.0, "cost": 0.0}
        port_totals[port]["qty"] += qty
        port_totals[port]["cost"] += qty * price

```

```

    port_average_prices = {
        port: totals["cost"] / totals["qty"]
        for port, totals in port_totals.items()
        if totals["qty"] > 0
    }
    external_price = max(port_average_prices.values()) *
    ↪ external_price_multiplier

```

```

return PriceData(
    exact_prices=exact_prices,
    port_average_prices=port_average_prices,
    external_price=external_price,
    external_price_multiplier=external_price_multiplier,
)

```

D.3 Kjøpsmuligheter per etappe. For hver etappe opprettes en kjøpsvariabel per priset modellhavn som er observert tilgjengelig i ruten. Eksakt månedspris brukes når den finnes, ellers historisk havnesnitt.

```

def build_purchase_opportunities(
    legs: list[Leg],
    price_data: PriceData,
) -> list[PurchaseOpportunity]:
    opportunities: list[PurchaseOpportunity] = []
    for leg_index, leg in enumerate(legs):
        for port in sorted(set(leg.available_ports)):
            if port not in price_data.port_average_prices:
                continue
            exact_price = price_data.exact_prices.get((leg.period_month, port))
            if exact_price is not None:
                opportunities.append(
                    PurchaseOpportunity(
                        leg_index=leg_index,
                        port=port,
                        price=exact_price,
                        price_source="monthly_observation",
                    )
                )
            else:
                opportunities.append(
                    PurchaseOpportunity(
                        leg_index=leg_index,
                        port=port,
                        price=price_data.port_average_prices[port],

```

```

        price_source="historical_port_average",
    )
)
return opportunities

```

D.4 Oppbygging og løsning av LP-problemet. Beslutningsvariablene er kjøp i priset havn (x), ekstern/ukjent bunkring (u) og beholdning etter hver etappe (I). Likhetsrestriksjonene er beholdningsbalansen, ulikhetsrestriksjonene er tankkapasiteten, og målfunksjonen minimerer samlet modellert kostnad. Problemet løses med `scipy.optimize.linprog` (HiGHS).

```

def solve_vessel(vessel_file_id, legs, capacity, price_data):
    opportunities = build_purchase_opportunities(legs, price_data)
    n_x = len(opportunities)
    n_l = len(legs)
    u_offset = n_x
    inv_offset = n_x + n_l
    n_vars = n_x + (2 * n_l)

    # Ikke-negative variabler; beholdning begrenses oppad av kapasitet.
    bounds: list[tuple[float, float | None]] = []
    bounds.extend((0.0, None) for _ in opportunities)
    bounds.extend((0.0, None) for _ in legs)
    bounds.extend((0.0, capacity) for _ in legs)

    # Likhet: beholdningsbalanse  $I_l = I_{l-1} + kjop + ekstern - forbruk$ .
    a_eq: list[list[float]] = []
    b_eq: list[float] = []
    for leg_index, leg in enumerate(legs):
        row = [0.0] * n_vars
        for opp_index, opp in enumerate(opportunities):
            if opp.leg_index == leg_index:
                row[opp_index] = 1.0
        row[u_offset + leg_index] = 1.0
        row[inv_offset + leg_index] = -1.0
        if leg_index > 0:
            row[inv_offset + leg_index - 1] = 1.0

```

```

        b_value = leg.consumption
    else:
        b_value = leg.consumption - leg.rob_start
    a_eq.append(row)
    b_eq.append(b_value)

# Ulikhet: beholdning for forbruk kan ikke overstige tankkapasitet.
a_ub_capacity: list[list[float]] = []
b_ub_capacity: list[float] = []
for leg_index, leg in enumerate(legs):
    row = [0.0] * n_vars
    for opp_index, opp in enumerate(opportunities):
        if opp.leg_index == leg_index:
            row[opp_index] = 1.0
    row[u_offset + leg_index] = 1.0
    if leg_index > 0:
        row[inv_offset + leg_index - 1] = 1.0
        b_value = capacity
    else:
        b_value = capacity - leg.rob_start
    a_ub_capacity.append(row)
    b_ub_capacity.append(b_value)

# Maalfunksjon: pris per kjøp pluss proxypris per ekstern/ukjent enhet.
objective = [0.0] * n_vars
for opp_index, opp in enumerate(opportunities):
    objective[opp_index] = opp.price
for leg_index in range(n_l):
    objective[u_offset + leg_index] = price_data.external_price

result = linprog(
    c=objective,
    A_ub=a_ub_capacity,
    b_ub=b_ub_capacity,
    A_eq=a_eq,

```

```
        b_eq=b_eq,  
        bounds=bounds,  
        method="highs",  
    )  
    if not result.success:  
        raise RuntimeError(f"LP feilet for {vessel_file_id}: {result.message}")  
    return result
```

Vedlegg D Sentral modellkode (utdrag fra run_route_inventory_model.py). Fullstendig kildekode ligger i 006 analysis.